

CS 698B: Fundamentals of Data Engineering (Part I)				Quiz 1 (19 Aug 2026)			
Name	DEEBO			20 marks Page 1 of 2			
Roll No	26007	Dept.	AWSM				

Instructions:

1. This question paper contains 1 page(s) (2 sides of paper). Please verify.
2. Write your name, roll number, department in **block letters** with **ink** on **each page**.
3. Write your final answers neatly **with a blue/black pen**. Pencil marks may get smudged.
4. Don't overwrite/scratch answers especially in MCQ – ambiguous cases will get 0 marks.



Q1. (BTM) Write the output of the code snippets. Each box must contain a single character. Leave one box blank to indicate a space. Use a new row for each line (if multiline output) starting from leftmost box. If output in a line does not need 8 boxes, leave remaining boxes **on the right** blank. *Hint: the Boolean literals True, False are interpreted as 1, 0 respectively if used in expressions where an integer is expected e.g. True + True + True = 3 and (4, 2) [False] = 4* (5 x 2 = 10 marks)

SNo.	Code Snippet	Output																								
1	<pre>my_str1 = "banana" my_str2 = "anbana" my_zip = zip(my_str1, my_str2) my_list = ["x="[a == b] for (a, b) in my_zip] # An empty string is calling join verdict = "".join(my_list) print(my_str1) # Do not leave boxes on the left print(my_str2) # blank unnecessarily. Leave print(verdict) # unwanted boxes blank on the right</pre>	<table><tr><td>b</td><td>a</td><td>n</td><td>a</td><td>n</td><td>a</td><td></td><td></td></tr><tr><td>a</td><td>n</td><td>b</td><td>a</td><td>n</td><td>a</td><td></td><td></td></tr><tr><td>x</td><td>x</td><td>x</td><td>=</td><td>=</td><td>=</td><td></td><td></td></tr></table>	b	a	n	a	n	a			a	n	b	a	n	a			x	x	x	=	=	=		
b	a	n	a	n	a																					
a	n	b	a	n	a																					
x	x	x	=	=	=																					
2	<pre>num = True + False for _ in range(ord('D') - ord('A')): # a <= b is a compound operator for a = a << b num <= num print(num)</pre>	<table><tr><td>2</td><td>0</td><td>4</td><td>8</td><td></td><td></td><td></td><td></td></tr></table>	2	0	4	8																				
2	0	4	8																							
3	<pre>s = "minion" # string is interpreted as int in base 2 (binary) m = int("10" * 2, base = 2) u = len(s) - 1 l = -len(s) * m - 1 total = 0 for i in range(u, l, -1): total += (s[i % len(s)] == s[True]) print(total)</pre>	<table><tr><td>2</td><td>2</td><td></td><td></td><td></td><td></td><td></td><td></td></tr></table>	2	2																						
2	2																									
4	<pre>my_str = "hello" for _ in range(True + True): # join treats strings as a list of characters my_str = "x".join(my_str) print(my_str[:8])</pre>	<table><tr><td>h</td><td>x</td><td>x</td><td>x</td><td>e</td><td>x</td><td>x</td><td>x</td></tr></table>	h	x	x	x	e	x	x	x																
h	x	x	x	e	x	x	x																			
5	<pre>a = 0 b = [a, [a]] c = b d = b.copy() b[0] = 1 b[1][0] = 2 a = 3 # print places a single space b/w outputs by default print(b[0], b[1][0]) print(c[0], c[1][0]) print(d[0], d[1][0])</pre>	<table><tr><td>1</td><td></td><td>2</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>1</td><td></td><td>2</td><td></td><td></td><td></td><td></td><td></td></tr><tr><td>0</td><td></td><td>2</td><td></td><td></td><td></td><td></td><td></td></tr></table>	1		2						1		2						0		2					
1		2																								
1		2																								
0		2																								

Q2. (CTC) A valid email address looks like `local@domain` and contains exactly one `@`, `local` and `domain` are non-empty strings containing only lowercase letters and dots. For example, the domain in `dee.bo@iitk.ac.in` is `iitk.ac.in`. Deebo wrote the following code to extract the domain from a valid email address, but Deebo spilled coffee on it erasing some of the parts. Complete the code by specifying what to write in the blank spaces `#A#`, `#B#` and `#C#`. We will replace `#A#` (including the `#`) with the completion you specify, similarly for `#B#` and `#C#`. The code will then be graded against hidden test cases. No marks if code does not run (e.g. if either completion is blank etc). (5 marks)

```
email = input( "Enter a valid email address: " )
domain = [ #A# for #B# if #C# ][0]
print( domain )
```

There are multiple ways to solve this problem. Grading will be based on hidden test cases.

Blank	Your completion	Alternate completions
#A#	<code>email[i + 1:]</code>	<code>email[email.index('@') + 1:]</code> <code>email.split('@')[1]</code>
#B#	<code>(i, c) in enumerate(email)</code>	<code>_ in range(1)</code>
#C#	<code>c == '@'</code>	<code>True</code>

Q3. (BTJ) Deebruh runs a grocery store and has collected customer purchase data as a list. Each list element is a *valid record* – a 3 tuple (`name`, `item`, `qty`) where `name`, `item` are non-empty strings containing only lowercase letters and `qty` is a strictly positive integer (cannot be 0). Duplicate tuples are allowed in the list (since customers may repeat purchase). Deebruh wrote the following code to print a sorted list of customer names who have purchased at least two distinct items in the past (item quantity is irrelevant) but the code is buggy. Specify a test case which reveals the fault:

1. Your test case must be a list of exactly 5 valid records (see definition of valid record above)
2. The correct output for your test case must be an empty list (no customer qualifies)
3. Deebruh's buggy code must produce a list of exactly 2 names for your test case

No marks if your test case causes the (buggy/correct) code to crash or give no output. (5 marks)

There are multiple ways to design the fault-revealing test case. Any solution that has at least two distinct customers each of which buys the same item at least twice (quantity irrelevant so long as strictly positive) will trigger the bug. There may or may not be a third customer.

Deebruh's buggy code	Your test case
<pre>records = [...] # Your test case purchases = {} # Empty dict for name, item, qty in records: if name not in purchases: purchases[name] = [] purchases[name].append(item) names = [name for name in purchases if len(purchases[name]) >= 2] names.sort() print(names)</pre>	<pre>[name↓ item↓ qty↓ ("deebo", "cake", 1), ("deebo", "cake", 1), ("deebo", "cake", 1), ("deebo", "cake", 1), ("deebo", "cake", 1), ("melbo", "cake", 1),]</pre>